SYSTEM DESIGN DOCUMENT

REQUI

Enterprise Healthcare Compliance AI Operating System
Master Project Specification

Author: Angelo Senat, Product Architect & Prompt Engineer

Version: 1.0

Date: April 13, 2026

Classification: Internal — Developer Ready

Table of Contents

1. System Architecture Overview

1.1 End-to-End Flow

1.2 Agent Orchestration

1.3 Data Lifecycle

2. Core Ontology (Data Model)

2.1 Core Objects

2.2 Object Relationships

3. Agent Definitions

3.1 Contract Agent

3.2 Compliance Agent

3.3 Gap & Risk Agent

3.4 Execution Agent

4. Backend Architecture

5. Database Schema

6. API Endpoints

7. AI System Prompts (Claude)

8. Scoring Engine

9. Gap Detection Engine

10. Task Generation Engine

11. Event-Driven System

12. UI / UX Direction

13. Enterprise Features

14. Deployment Requirements

15. Implementation Roadmap

1. System Architecture Overview

> **Core Principle**
>
> REQUI is an AI-driven compliance + operations engine — NOT a chatbot, NOT a dashboard tool, NOT a generic SaaS. It transforms contracts, compliance data, and user input into answers, risks, gaps, tasks, and deadlines.

1.1 End-to-End Flow

The system enforces a mandatory pipeline that processes every input through a standardized sequence:

1. **Input** — User query or document upload
2. **Contract Parsing** — Extract obligations (if applicable)
3. **Compliance Normalization** — Map to regulatory framework
4. **Gap Detection** — Identify missing compliance elements
5. **Task Generation** — Convert gaps into actionable tasks
6. **Score Update** — Recalculate compliance metrics
7. **Response Output** — Deliver structured results to user

1.2 Agent Orchestration

The system is structured around four specialized agents, each with a single, well-defined responsibility:

Table 1: Agent System Structure

Agent	Role	Primary Function
Contract Agent	Extractor	Extracts obligations, deadlines, and requirements from contracts
Compliance Agent	Mapper	Maps extracted obligations to regulatory frameworks

Gap & Risk Agent	Detector	Identifies missing compliance elements and calculates risk exposure
Execution Agent	Actor	Converts gaps into tasks, reminders, and actionable items

1.3 Data Lifecycle

Data flows through the system following a consistent lifecycle pattern:

Entry Points

- **User Prompt:** Direct question (e.g., "What do I need to report?")
- **Contract Upload:** PDF or CLM data (Juro, Ironclad, etc.)

Processing Pipeline

1. **Ingestion:** Parse contract/input, extract raw text, store in Documents
2. **Contract Agent:** Extract obligations, deadlines, reporting requirements to Requirements object
3. **Compliance Agent:** Pull from federal rules, state rules, compliance database to normalize requirements
4. **Gap & Risk Agent:** Compare Required vs Existing to output missing items, risk score inputs, audit gaps
5. **Execution Agent:** Convert gaps to tasks, deadlines, reminders
6. **Scoring Engine:** Update compliance score, risk score, audit readiness
7. **Dashboard Update:** Push tasks, gaps, scores, alerts to UI

2. Core Ontology (Data Model)

All system data maps to a unified ontology. Every object, relationship, and attribute is explicitly defined to ensure consistency across the platform.

2.1 Core Objects

Table 2: Core Data Objects

Object	Description	Key Attributes
Organization	Healthcare entity (provider, MSO, payer)	id, name, type, states[], created_at
User	System user with role-based access	id, organization_id, email, role
Contract	Legal agreement with compliance obligations	id, org_id, document_id, name, dates
Requirement	Compliance obligation (extracted or regulatory)	id, org_id, source, category, description
Task	Actionable item derived from gaps	id, org_id, requirement_id, title, status
Deadline	Time-bound compliance milestone	id, requirement_id, due_date, frequency
Compliance Gap	Identified deficiency in compliance posture	id, org_id, requirement_id, severity
Score	Quantified compliance metrics	id, org_id, compliance, risk, audit scores
Document	Uploaded file with parsed content	id, org_id, type, file_url, parsed_text

2.2 Object Relationships

Relationship Map

- **Contract** to Requirements (one-to-many)
- **Requirement** to Tasks (one-to-many)
- **Requirement** to Deadlines (one-to-many)
- **Requirement** to Compliance Gaps (one-to-many)
- **Task** to Score Impact (indirect via completion)
- **Organization** to All Objects (one-to-many)

Entity Relationship Diagram (Logical)

```
Organization (1)
    ├── Users (N)
    ├── Documents (N)
    │   └── Contracts (1:1)
    │       └── Contract_Requirements (N)
    │           └── Requirements (N)
    ├── Requirements (N)
    │   ├── Tasks (N)
    │   ├── Deadlines (N)
    │   └── Compliance_Gaps (N)
    └── Scores (N - history)
```

3. Agent Definitions

Each agent has a precisely defined role, input/output specification, operational rules, and failure handling procedures.

3.1 Contract Agent

Role

Extract obligations, deadlines, and reporting requirements from contract documents

Inputs

Document text (parsed PDF or CLM export), document_id, organization_id

Outputs

Structured array of extracted requirements with metadata (category, obligation_type, frequency, deadline, confidence)

Rules

- Extract ONLY explicit obligations from contract text
- Ignore vague or aspirational language
- Categorize into: FWA, HIPAA, Reporting, Quality, Claims/Encounter, General Compliance
- Assign confidence score (high/medium/low) based on text clarity
- Never hallucinate requirements not present in source

Failure Handling

- If parsing fails: Return empty requirements array with confidence: "low"
- If text is unreadable: Log error, request re-upload
- If ambiguous language: Flag for manual review

3.2 Compliance Agent

Role

Map extracted requirements to standardized healthcare compliance frameworks

Inputs

Extracted requirements array from Contract Agent, organization context (states, plan types)

Outputs

Normalized requirements with standardized categories, compliance domains, and priority levels

Rules

- Map to known frameworks: Federal (CMS, HIPAA, OIG), Medicaid Managed Care (42 CFR 438), State-level
- Assign priority: critical (regulatory required), high (contractual required), medium (operational), low (best practice)
- Do NOT invent new requirement categories
- If unclear mapping: mark confidence: "low"

Failure Handling

- If no framework match: Pass through with "unmapped" status
- If conflicting mappings: Return multiple options with confidence scores

3.3 Gap & Risk Agent

Role

Detect compliance gaps and calculate risk exposure by comparing requirements against organizational state

Inputs

Normalized requirements, existing tasks, documents, deadlines, completion status

Outputs

Gap list (with type, severity, reason, recommended action) + Risk summary (score, level, drivers)

Rules

- Evaluate each requirement for: Exists (yes/no), Documented (full/partial/none), Tracked (yes/no), Overdue (yes/no)
- Severity levels: critical (regulatory violation risk), high (required but incomplete), medium (partially implemented), low (improvement)
- Risk score calculation: missing critical (+25), no documentation (+20), overdue (+15), repeated gaps (+10), cap at 100

Failure Handling

- If insufficient data: Return gaps with "insufficient_data" flag
- If calculation error: Use conservative (higher) risk estimate

3.4 Execution Agent

Role

Convert compliance gaps into structured operational tasks and actions

Inputs

Detected gaps array with severity and recommended actions

Outputs

Structured tasks with title, description, action_type, priority, category, due_date, linked_requirement

Rules

- Map gaps to action types: REQUIRED_TASK, DOCUMENT_REQUEST, REVIEW_TASK, INVESTIGATION_TASK
- Priority mapping: critical gap to CRITICAL, high gap to HIGH, medium to MEDIUM, low to LOW
- Due date logic: if deadline exists use it; if regulatory default 30 days; if unknown null

Failure Handling

- If gap cannot be converted: Create generic "Review Required" task
- If duplicate task detected: Update existing instead of creating new

4. Backend Architecture

4.1 Service Layer

The backend consists of five core microservices plus supporting infrastructure:

Table 3: Backend Services

Service	Function	Technology
contract-agent-service	Calls Claude with Contract Agent prompt, extracts requirements	Node.js / Python FastAPI
compliance-agent-service	Calls Claude with Compliance prompt, normalizes requirements	Node.js / Python FastAPI
gap-agent-service	Calls Claude with Gap prompt, detects gaps and calculates risk	Node.js / Python FastAPI
execution-agent-service	Calls Claude with Execution prompt, generates tasks	Node.js / Python FastAPI
orchestrator-service	Controls flow, calls all agents in order, writes to DB	Node.js / Python FastAPI
score-engine-service	Recalculates compliance, risk, and audit scores	Node.js / Python

4.2 API Layer

RESTful API with optional GraphQL layer for complex queries. All endpoints return structured JSON and follow consistent error handling patterns.

4.3 Database Usage

PostgreSQL with JSONB hybrid model for flexibility:

- **Relational tables:** Core entities (organizations, users, contracts, requirements, tasks)
- **JSONB columns:** Flexible metadata, agent outputs, audit logs
- **Indexing:** GIN indexes on JSONB columns for query performance

4.4 Event-Driven Flows

Queue-based architecture (Kafka or AWS SQS) for asynchronous processing:

> **Event Flow**
>
> document_uploaded to contract_parsed to requirements_created to gaps_detected to tasks_generated to score_updated

5. Database Schema

5.1 Core Tables

organizations

Field	Type	Description
`id` PK	UUID	Primary identifier
`name`	TEXT	Organization name
`type`	TEXT	provider, MSO, payer
`states`	TEXT[]	Operating states
`created_at`	TIMESTAMP	Creation timestamp

users

Field	Type	Description
`id` PK	UUID	Primary identifier
`organization_id` FK	UUID	Link to organization
`email`	TEXT	User email (unique)
`role`	TEXT	admin, compliance_officer, viewer
`created_at`	TIMESTAMP	Creation timestamp

documents

Field	Type	Description
`id` PK	UUID	Primary identifier
`organization_id` FK	UUID	Link to organization
`type`	TEXT	contract, policy, etc.
`file_url`	TEXT	S3 storage URL

parsed_text	TEXT	Extracted text content
metadata	JSONB	Additional document info
created_at	TIMESTAMP	Upload timestamp

contracts

Field	Type	Description
id PK	UUID	Primary identifier
organization_id FK	UUID	Link to organization
document_id FK	UUID	Link to source document
contract_name	TEXT	Contract identifier
effective_date	DATE	Contract start date
expiration_date	DATE	Contract end date
metadata	JSONB	Contract metadata

requirements

Field	Type	Description
id PK	UUID	Primary identifier
organization_id FK	UUID	Link to organization
source	TEXT	federal, state, contract
category	TEXT	HIPAA, FWA, Reporting, etc.
description	TEXT	Requirement details
obligation_type	TEXT	reporting, policy, etc.
frequency	TEXT	monthly, quarterly, annual
metadata	JSONB	Additional requirement data

contract_requirements (Link Table)

Field	Type	Description

id PK	UUID	Primary identifier
contract_id FK	UUID	Link to contract
requirement_id FK	UUID	Link to requirement
extracted	BOOLEAN	AI-extracted flag

tasks

Field	Type	Description
id PK	UUID	Primary identifier
organization_id FK	UUID	Link to organization
requirement_id FK	UUID	Link to requirement
title	TEXT	Task title
description	TEXT	Task details
priority	TEXT	CRITICAL, HIGH, MEDIUM, LOW
status	TEXT	pending, in_progress, complete
due_date	TIMESTAMP	Task deadline
assigned_to FK	UUID	Assigned user
source	TEXT	AI, manual
created_at	TIMESTAMP	Creation timestamp

deadlines

Field	Type	Description
id PK	UUID	Primary identifier
requirement_id FK	UUID	Link to requirement
due_date	TIMESTAMP	Deadline timestamp
frequency	TEXT	recurring pattern
status	TEXT	upcoming, due, overdue

compliance_gaps

Field	Type	Description
`id` PK	UUID	Primary identifier
`organization_id` FK	UUID	Link to organization
`requirement_id` FK	UUID	Link to requirement
`severity`	TEXT	critical, high, medium, low
`status`	TEXT	open, in_progress, resolved
`evidence`	JSONB	Gap detection evidence
`created_at`	TIMESTAMP	Detection timestamp

scores

Field	Type	Description
`id` PK	UUID	Primary identifier
`organization_id` FK	UUID	Link to organization
`compliance_score`	INT	0-100 compliance percentage
`risk_score`	INT	0-100 risk exposure
`audit_score`	INT	0-100 audit readiness
`updated_at`	TIMESTAMP	Last calculation time

agent_logs (Audit Trail)

Field	Type	Description
`id` PK	UUID	Primary identifier
`agent_type`	TEXT	contract, compliance, gap, execution
`input`	JSONB	Agent input data
`output`	JSONB	Agent output data
`status`	TEXT	success, error, partial

`created_at`	TIMESTAMP	Execution timestamp

6. API Endpoints

6.1 Authentication

POST `/auth/login`

Description: Authenticate user and return JWT token

Response: `{ token: string, user: UserObject }`

6.2 Organization Management

GET `/organization/{id}`

Description: Retrieve organization details with current scores

POST `/organization`

Description: Create new organization

6.3 Document and Contract Ingestion

POST `/documents/upload`

Description: Upload file, returns document_id

Body: multipart/form-data with file

Response: `{ document_id: UUID, status: string }`

POST `/documents/{id}/parse`

Description: Trigger document parsing engine

POST /contracts/create

Description: Link document to contract record

6.4 Agent Endpoints

POST /agents/contract/extract

Description: Run Contract Agent on document

```
{ "document_id": "uuid" }
// Output: extracted obligations, deadlines, clauses
```

POST /agents/compliance/map

Description: Normalize requirements to compliance framework

```
{ "requirements": [...] }
// Output: normalized requirements with categories
```

POST /agents/gaps/analyze

Description: Detect compliance gaps for organization

```
{ "organization_id": "uuid" }
// Output: list of gaps with severity
```

POST /agents/tasks/generate

Description: Generate tasks from detected gaps

```
{ "gaps": [...] }
// Output: tasks created
```

6.5 Task Management

GET `/tasks`

Query params: organization_id, status, priority, assigned_to

POST `/tasks`

Description: Create manual task

PATCH `/tasks/{id}`

Description: Update task status, assignment, etc.

6.6 Scores

GET `/scores/{organization_id}`

Description: Get current compliance, risk, and audit scores

POST `/scores/recalculate`

Description: Trigger score recalculation

6.7 AI Interface (Main Entry Point)

POST /ai/query

Description: Primary AI interface — orchestrates all agents

```
{
  "organization_id": "uuid",
  "query": "What reporting do I need?",
  "document_text": "optional contract text"
}
```

Backend Flow: Compliance Agent to Contract Agent to Gap Agent to Task Agent

```
{
  "answer": "...",
  "requirements": [...],
  "gaps": [...],
  "tasks": [...],
  "risk_score": 72,
  "scores": {
    "compliance": 73,
    "audit": 62
  }
}
```

7. AI System Prompts (Claude)

Global Rules (Apply to All Agents)

- Do NOT hallucinate — only use provided input and validated compliance database
- Do NOT guess — if unsure, return `"confidence": "low"`
- Output MUST be valid JSON — no conversational text, no explanations outside JSON
- Always include: `"confidence"` and `"source_type"` (contract/federal/state/inferred)

7.1 Contract Agent System Prompt

```
SYSTEM PROMPT — Contract Extraction Agent

You are the Contract Extraction Agent for an enterprise healthcare compliance system. Your job is to read structured or unstructured contract text and extract ONLY: - reporting obligations - compliance requirements - deadlines - frequencies - regulatory references (if present) OUTPUT FORMAT: { "requirements": [ { "description": "", "category": "", "obligation_type": "", "frequency": "", "deadline": "", "regulatory_reference": "", "source_type": "contract", "confidence": "" } ] } RULES: - Extract only explicit obligations - Ignore vague language - If frequency not stated: null - If deadline not stated: null - Do not infer beyond text - Categorize into: FWA, HIPAA, Reporting, Quality, Claims/Encounter, General Compliance
```

7.2 Compliance Agent System Prompt

```
SYSTEM PROMPT — Compliance Intelligence Agent

You are the Compliance Intelligence Agent. Your job is to take extracted requirements and map them into standardized healthcare compliance categories using known frameworks: - Federal (CMS, HIPAA, OIG) - Medicaid Managed Care (42 CFR 438) - State-level compliance OUTPUT FORMAT: { "normalized_requirements": [ { "original_description": "", "standardized_category": "", "compliance_domain": "", "reporting_type": "", "priority_level": "", "source_type": "", "confidence": "" } ] } RULES: - Do NOT invent requirements - Map only to known categories - Assign priority: * critical (regulatory required) * high (contractual required) * medium (operational) * low (best practice) - If unclear mapping: mark "confidence": "low"
```

7.3 Gap & Risk Agent System Prompt

```
SYSTEM PROMPT — Gap & Risk Detection Agent

You are the Gap & Risk Detection Agent. Your job is to compare: - required compliance obligations VS - organization state (tasks, documents, deadlines) INPUT EXPECTATION: { "requirements": [...], "documents": [...], "tasks": [...], "deadlines": [...] } OUTPUT FORMAT: { "gaps": [ { "requirement_id": "", "gap_type": "", "severity": "", "reason": "", "recommended_action": "", "confidence": "" } ], "risk_summary": { "risk_score": 0, "risk_level": "", "drivers": [], "confidence": "" } } GAP LOGIC: Evaluate for each requirement: - missing: no task, no document - partial: incomplete evidence - overdue: deadline passed - untracked: no system linkage SEVERITY RULES: - critical: regulatory violation risk - high: required but incomplete - medium: partially implemented - low: improvement RISK SCORE LOGIC: - missing critical: +25 - no documentation: +20 - overdue: +15 - repeated gaps: +10 Cap at 100
```

7.4 Execution Agent System Prompt

```
SYSTEM PROMPT — Execution Agent

You are the Execution Agent. Your job is to convert compliance gaps into structured operational tasks. OUTPUT FORMAT: { "tasks": [ { "title": "", "description": "", "action_type": "", "priority": "", "category": "", "due_date": "", "linked_requirement": "", "confidence": "" } ] } RULES: Map gaps to: - REQUIRED_TASK - DOCUMENT_REQUEST - REVIEW_TASK - INVESTIGATION_TASK PRIORITY LOGIC: - critical gap: CRITICAL - high gap: HIGH - medium: MEDIUM - low: LOW DUE DATE LOGIC: - if deadline exists: use it - if regulatory: default 30 days - if unknown: null
```

7.5 AI Orchestrator System Prompt

```
SYSTEM PROMPT — AI Orchestrator Agent

You are the AI Orchestrator Agent. You do NOT answer directly. You coordinate: 1. Contract Agent 2. Compliance Agent 3. Gap Agent 4. Execution Agent FLOW: - If input = contract: send to Contract Agent - Map output: Compliance Agent - Send to Gap Agent - Send to Execution Agent FINAL OUTPUT FORMAT: { "answer": "", "requirements": [...], "gaps": [...], "tasks": [...], "risk_score": 0, "confidence": "" } RULES: - Combine outputs - Do NOT invent new data - If missing data: flag low confidence - Keep answer short + structured
```

8. Scoring Engine

The scoring engine calculates three key metrics that quantify organizational compliance posture. Scores are recalculated automatically on relevant events.

8.1 Compliance Score

```
Compliance Score = (Requirements Satisfied / Total Requirements) x 100
```

A requirement is "satisfied" when:

- Associated task exists and is marked complete, OR
- Required documentation is present and verified, OR
- Deadline is met with evidence of completion

8.2 Risk Score

```
Risk Score = Sum of Weighted Gap Severities + Overdue Penalties + Documentation Penalties
```

Component breakdown:

Factor	Weight	Condition
Missing Critical	+25	Critical requirement with no task/document
No Documentation	+20	Required documentation absent
Overdue Task	+15	Task past due date
Repeated Gap	+10	Same gap detected in consecutive periods

Maximum cap: 100. Risk levels: 0-25 (Low), 26-50 (Medium), 51-75 (High), 76-100 (Critical)

8.3 Audit Readiness Score

```
Audit Score = (Documentation Score x 0.4) + (Traceability Score x 0.35) + (Log Completeness x 0.25)
```

Components:

- **Documentation Score:** Percentage of requirements with complete, accessible documentation
- **Traceability Score:** Percentage of tasks linked to requirements with clear audit trail
- **Log Completeness:** Percentage of agent actions logged with input/output

8.4 Score Triggers

Event	Action
Task completed	Recalculate compliance, risk
Task overdue	Increase risk score
Document uploaded	Recalculate audit, compliance
Gap resolved	Decrease risk score
New requirement added	Full recalculation

9. Gap Detection Engine

The gap detection engine evaluates each requirement against four dimensions to identify compliance deficiencies.

9.1 Detection Logic

For each requirement, the engine evaluates:

Dimension	Check	Gap Type if Failed
Exists	Is there evidence of compliance?	MISSING
Documented	Is documentation complete?	PARTIAL
Tracked	Is there a task or deadline tracking?	UNTRACKED
Overdue	Has the deadline passed?	OVERDUE

9.2 Gap Classification

Gap Type	Definition	Example
MISSING	No task, no document, no evidence of compliance	Required quarterly report never created
PARTIAL	Documentation exists but incomplete or insufficient	Policy document missing required sections
UNTRACKED	Compliance activity exists but not in system	Report submitted but no task record
OVERDUE	Deadline passed without completion	Annual audit 30 days past due

9.3 Severity Assignment

Severity	Criteria	Action Required
CRITICAL	Regulatory violation risk; potential penalties	Immediate escalation
HIGH	Required but incomplete; contractual obligation	Address within 7 days
MEDIUM	Partially implemented; operational gap	Address within 30 days

LOW	Improvement opportunity; best practice	Address when resources allow

10. Task Generation Engine

The task generation engine converts detected gaps into actionable, trackable tasks with appropriate prioritization and deadlines.

10.1 Gap-to-Task Mapping Rules

Gap Type	Action Type	Task Template
MISSING	REQUIRED_TASK	"Create [deliverable] for [requirement]"
PARTIAL	DOCUMENT_REQUEST	"Complete documentation for [requirement]"
UNTRACKED	REVIEW_TASK	"Verify and track [requirement] in system"
OVERDUE	INVESTIGATION_TASK	"Investigate overdue [requirement] — determine status"

10.2 Priority Logic

Gap Severity	Task Priority	SLA
CRITICAL	CRITICAL	24 hours
HIGH	HIGH	7 days
MEDIUM	MEDIUM	30 days
LOW	LOW	90 days

10.3 Due Date Logic

Due Date Assignment

- **If deadline exists:** Use the explicit deadline from requirement
- **If regulatory requirement:** Default to 30 days from detection
- **If unknown:** Set to null (requires manual assignment)
- **If overdue gap:** Set to immediate (today + 1 day)

10.4 Task Deduplication

Before creating a new task, the engine checks for existing tasks with:

- Same linked_requirement
- Same or higher priority
- Status not "complete"

If duplicate found: Update existing task instead of creating new.

11. Event-Driven System

The system uses an event-driven architecture with a message queue (Kafka or AWS SQS) to enable asynchronous processing and decouple components.

11.1 Event Definitions

Event	Trigger	Downstream Effects
`document_uploaded`	File uploaded to /documents/upload	Trigger parsing to contract_parsed
`contract_parsed`	Document text extraction complete	Trigger Contract Agent to requirements_created
`requirements_created`	New requirements inserted into DB	Trigger Gap Analysis to gaps_detected
`gaps_detected`	Gap analysis complete	Trigger Task Generation to tasks_generated
`tasks_generated`	New tasks created	Trigger Score Recalculation to score_updated
`task_completed`	Task status changed to "complete"	Update requirement to recalculate scores
`deadline_missed`	Scheduled check detects overdue deadline	Increase risk score to alert user
`score_updated`	Score recalculation complete	Push update to UI dashboard

11.2 Event Schema

```
{
  "event_id": "uuid",
  "event_type": "document_uploaded",
  "organization_id": "uuid",
  "payload": {
    "document_id": "uuid",
    "file_name": "contract.pdf",
    "uploaded_by": "uuid"
  },
  "timestamp": "2026-04-13T10:30:00Z",
  "correlation_id": "uuid"
}
```

11.3 Dead Letter Queue

Failed events are retried 3 times with exponential backoff, then moved to DLQ for manual review.

12. UI / UX Direction

Design Principles

- AI-first interface — Intelligence is the landing page
- Dashboard is secondary — Show only what matters
- Task + compliance integration — No separation
- No complex dashboards — Simple, fast, actionable

12.1 Primary Screen Layout

The main interface centers on AI interaction with supporting panels. The layout follows this structure:

- **Top Bar:** Logo, AI input field, user profile
- **Main Panel (60%):** AI response with structured output
- **Side Panel (40%):** Scores, deadlines, active tasks

12.2 Output Panel Structure

Every AI response follows the same structured format:

1. **Answer** — Direct response to user query
2. **Key Requirements** — Relevant compliance obligations
3. **Detected Risks** — Risk items with severity
4. **Compliance Gaps** — Missing elements with fix buttons
5. **Recommended Actions** — Tasks that can be created with one click

12.3 Side Panel Contents

- **Tasks:** Active tasks with quick-complete actions
- **Deadlines:** Upcoming and overdue deadlines
- **Scores:** Current compliance, risk, and audit scores
- **Documents:** Recently uploaded contracts and files

12.4 UX Rules

Rule	Implementation

No complex dashboards	Single-screen experience, no navigation depth
No data pipelines visible	Processing happens behind the scenes
No engineering UI	Business-friendly language, no technical jargon
One-click actions	Tasks created with single button press
Contextual help	Inline explanations, no separate help section

13. Enterprise Features

Enterprise-tier capabilities extend the core system for large healthcare organizations.

13.1 Contract Ingestion

Phase	Capability	Status
Phase 1	Manual PDF upload with text extraction	Core
Phase 2	CLM integrations (Juro, Ironclad, DocuSign CLM)	Add-on
Phase 3	Live sync + automatic re-analysis on contract changes	Enterprise

13.2 Multi-Organization Support

- Parent-child organization hierarchies
- Cross-org compliance reporting
- Consolidated dashboards for MSOs
- Organization-specific requirement filtering

13.3 Role-Based Access Control (RBAC)

Role	Permissions
Admin	Full access, user management, configuration
Compliance Officer	View all, create tasks, upload documents, resolve gaps
Analyst	View scores, tasks, run queries, cannot modify
Viewer	View-only access to dashboard and reports

13.4 Integrations

Category	Systems	Integration Type
Identity	Microsoft Azure AD, Okta	SSO / SAML
Storage	SharePoint, Google Drive	Document sync

Communication	Slack, Microsoft Teams	Alert notifications
Calendar	Outlook, Google Calendar	Deadline sync
CLM	Juro, Ironclad, DocuSign	API webhooks

14. Deployment Requirements

14.1 API Keys and Services

Service	Purpose	Required
Anthropic Claude API	AI agent processing	Required
AWS S3 / GCS	Document storage	Required
PostgreSQL	Primary database	Required
Redis	Caching, session store	Recommended
Kafka / AWS SQS	Event queue	Required
Auth0 / Cognito	Authentication	Required

14.2 Backend Stack

Component	Technology	Alternative
API Server	Node.js (Express/NestJS)	Python (FastAPI)
Database	PostgreSQL 14+	—
Queue	Apache Kafka	AWS SQS, RabbitMQ
Cache	Redis	Memcached
Storage	AWS S3	Google Cloud Storage
AI Client	Anthropic SDK	REST API direct

14.3 Infrastructure Basics

- **Containerization:** Docker + Kubernetes (EKS/GKE)
- **CI/CD:** GitHub Actions to Docker build to K8s deploy
- **Monitoring:** Datadog or New Relic for APM
- **Logging:** ELK stack or CloudWatch
- **Security:** TLS 1.3, encryption at rest, SOC 2 compliance

14.4 Environment Variables

```
CLAUDE_API_KEY=sk-ant-...
DATABASE_URL=postgresql://...
REDIS_URL=redis://...
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
S3_BUCKET_NAME=requi-documents
KAFKA_BROKERS=...
JWT_SECRET=...
AUTH0_DOMAIN=...
```

15. Implementation Roadmap

15.1 Phase 1: Foundation

Phase 1 — Core Pipeline (Weeks 1-6)

Goal: Establish core ontology and basic contract parsing pipeline

Deliverable	Description	Priority
Core Ontology	Database schema, object definitions, relationships	P0
Contract Parser	PDF text extraction, Contract Agent integration	P0
Basic Pipeline	/documents/upload to /ai/query end-to-end flow	P0
Auth System	JWT-based authentication, user management	P1

15.2 Phase 2: Intelligence

Phase 2 — Gap Detection and Task Generation (Weeks 7-12)

Goal: Implement full gap detection, task generation, and scoring

Deliverable	Description	Priority
Gap Detection Engine	Gap & Risk Agent, severity classification	P0
Task Generation	Execution Agent, task templates, deduplication	P0
Scoring Engine	Compliance, risk, audit score calculations	P1
Event System	Queue integration, event handlers	P1

15.3 Phase 3: Scale

Phase 3 — Integrations and Enterprise (Weeks 13-18)

Goal: Add enterprise features, integrations, and UI refinement

Deliverable	Description	Priority
CLM Integrations	Juro, Ironclad, DocuSign CLM connectors	P2
Multi-Org Support	Organization hierarchies, cross-org reporting	P2
RBAC	Role-based access control, permissions	P1
UI Refinement	Polished interface, responsive design	P2
Audit Logging	Complete agent_logs implementation	P1

15.4 Implementation Priority Summary

Start With

1. Ontology (data model)
2. Contract Agent + parsing
3. Basic pipeline (/ai/query)
4. Gap Detection Engine
5. Task Generation
6. Scoring Engine
7. UI integration
8. Logging system

15.5 Success Metrics

Metric	Target	Measurement
Contract parsing accuracy	greater than 90%	Manual review sample

Gap detection precision	greater than 85%	False positive rate
Task creation relevance	greater than 90%	User acceptance rate
API response time	less than 5s	p95 latency
System uptime	greater than 99.9%	Monitoring dashboard

Final Architecture Principle

> **REQUI = Intelligence + Action + Simplicity**
>
> Each agent does ONE job, outputs structured JSON, passes to the next agent. The backend guarantees: Every AI call = structured output + DB update.

User Experience Goal

1. Upload contract or ask question
2. System analyzes instantly
3. See what's required, what's missing, what to do next

No thinking required. No manual tracking. Always audit-aware.

REQUI — Enterprise Healthcare Compliance AI Operating System

Version 1.0 | April 13, 2026 | Internal — Developer Ready